

LLM 中文文本生成系统

用户操作手册

V1.0

编制日期：2026 年 9 月

文档说明

本手册面向教师、编辑、研究人员和系统维护人员，说明软件的安装、配置、内容生成、质量评估、批量处理、模型微调 and 结果查看方法。基础模式无需网络、显卡或外部模型即可完成全部 workflow 演示。

项目	说明
软件名称	LLM 中文文本生成系统
版本	V1.0
运行方式	命令行应用
基础环境	Python 3.10 及以上
默认模式	离线模板生成与规则质量评估
可选能力	本地 Transformers 模型生成与 LoRA 微调

目录

- 1 软件概述
- 2 安装与启动
- 3 快速演示
- 4 单篇内容生成
- 5 文本质量评估
- 6 批量任务处理
- 7 训练数据准备
- 8 LoRA 模型微调
- 9 配置说明
- 10 输出文件说明
- 11 一键功能测试

- 12 Notebook 分批测试
- 13 常见问题与故障处理
- 14 数据安全与使用边界
- 15 日常维护与卸载
- 附录 A 命令速查
- 附录 B 验收检查表

1 软件概述

1.1 软件用途

系统围绕小学、初中和高中三个阅读等级，完成中文阅读材料生成、自动质量检查和批量报告导出。它解决原始研究脚本入口分散、路径依赖、环境配置不一致和结果难以汇总的问题，把模型生成、文本统计和分级评估组织为可重复执行的软件流程。

1.2 核心功能

- 按主题、学段、关键词和附加要求生成中文阅读材料。
- 使用离线模板后端完成安装验收，或接入用户拥有的本地语言模型。
- 计算汉字数、句数、段落数、平均句长、重复率、英文比例和难度匹配度。
- 按优秀、良好、合格、待改进给出综合结论，并列出警告和修改建议。
- 从 JSONL 读取批量任务，隔离单条异常并输出 HTML、CSV、JSON 和 JSONL 报告。
- 把带学段标签的文本转换为训练数据，并对兼容模型执行 LoRA 微调。

1.3 工作流程

用户先选择生成后端并给出主题。系统校验请求后生成正文，将正文交给评估器计算统计指标和综合分，再把文章、分数、提示与元数据写入结构化结果。批量模式会重复这一流程，并在结束后生成汇总报告。

2 安装与启动

2.1 环境要求

类别	最低要求	建议配置
操作系统	Windows 10、常见 Linux 或 macOS	64 位桌面或服务器系统
Python	3.10	3.11 或 3.12
内存	4 GB	8 GB 以上
磁盘	基础软件 50 MB 内	按本地模型体积预留空间
显卡	基础模式不需要	模型模式按模型规模配置 CUDA 显卡

2.2 安装基础软件

在命令行进入软件根目录，即包含 `pyproject.toml` 的目录，执行以下命令。安装过程只安装本软件基础包，默认不下载大模型。

```
python -m pip install -e .  
llmctg --version
```

看到 `llmctg 1.0.0` 表示入口安装成功。若不执行安装，也可以将 `src` 目录加入 `PYTHONPATH` 后使用 `python -m llmctg`。

2.3 安装模型扩展

只有需要加载本地 Hugging Face 模型或执行 LoRA 微调时才安装扩展依赖。模型权重不包含在本软件中，应由用户从合法来源准备。

```
python -m pip install -e "[llm]"
```

2.4 Linux GPU 环境检查

在加载模型前，应确认驱动、PyTorch 和 CUDA 运行时能够协同工作。显卡驱动显示的 `CUDA Version` 表示驱动可支持的最高 CUDA 版本，不等同于当前 PyTorch 的编译版本。以下命令用于核对设备状态。

```
nvidia-smi
python -c "import torch; print(torch.__version__); print(torch.version.cuda);
print(torch.cuda.is_available()); print(torch.cuda.get_device_name(0))"
输出中的 torch.cuda.is_available() 应为 True，并能显示显卡名称。若服务器拥有多张卡，可在命
```

令前设置 CUDA_VISIBLE_DEVICES=0，只向当前进程暴露一张物理卡。此时程序内部仍使用 cuda:0。

2.5 Linux 预设部署路径

配套测试脚本默认使用下列部署路径。路径不一致时，可以通过脚本参数覆盖，也可以在 Notebook 的初始化单元中修改。

用途	预设路径
软件目录	/home/wangzy/workplace/wyc/llmctg
基础模型	/home/wangzy/workplace/wyc/Qwen3-1.7B
训练样本	test_data/llmctg_smoke_train.txt
测试输出	test_output/时间戳目录

3 快速演示

快速演示不调用外部模型，会分别生成小学、初中和高中示例，完成自动评分并输出报告。

```
llmctg demo --output output/demo
```

文件	用途
articles.jsonl	逐条保存文章、请求信息和完整评估结果
summary.json	保存数量、通过数、平均分和异常数
summary.csv	用于办公软件筛选和统计
report.html	用于浏览器阅读文章和质量提示

4 单篇内容生成

4.1 基本命令

```
llmctg generate --topic "海洋" --level junior --count 2 --seed 2026 --output  
output/ocean.jsonl
```

参数	是否必填	说明
--topic	是	文章主题，不允许为空
--level	否	primary、junior、senior 或中文学段，默认 junior
--instruction	否	补充内容范围、体裁或表达要求
--keywords	否	用中文或英文逗号分隔
--count	否	一次生成 1 至 20 篇
--seed	否	固定随机种子以便复现
--backend	否	template 或 transformers
--output	否	JSONL 文件；省略时打印到终端

4.2 指定本地模型

```
llmctg generate --backend transformers --model-path D:/models/Qwen --device auto --topic "  
城市生态" --level senior
```

系统会在首次生成时加载模型。**device** 为 **auto** 时优先使用可用 **CUDA**，否则使用 **CPU**。为降低供应链风险，软件不会执行模型目录中的远程自定义代码。

4.3 加载 LoRA 适配器生成

微调完成后，使用 **--lora-path** 指向包含 **adapter_config.json** 和 **adapter_model.safetensors** 的适配器目录。系统先加载基础模型，再以只读推理方式挂载适配器。基础模型必须与训练该适配器时使用的模型一致。

```
CUDA_VISIBLE_DEVICES=0 llmctg generate \  
--backend transformers \  
--lora-path D:/models/Qwen/llmctg_adapter/
```

```
--model-path /home/wangzy/workplace/wyc/Qwen3-1.7B \
--lora-path test_output/qwen3-1.7b-lora-smoke \
--device cuda:0 \
--topic "城市河流与生态保护" \
--level junior \
--instruction "说明生态价值、污染来源和保护方法" \
--keywords "河流,生态,污染,保护" \
--count 1 --seed 2026 \
--output test_output/qwen3-lora-generation.jsonl
```

命令中的反斜杠只放在 Linux 命令行的行末用于续行。参数名前应写 `--`，变量名中不要插入反斜杠。例如应写 `--model-path` 和 `$MODEL_PATH`，不能写成 `\--model-path` 或 `$MODEL\_PATH`。

4.4 生成参数的影响

参数	作用	使用建议
temperature	控制采样随机性	较低值更稳定，较高值更多样
top_p	限制累计概率范围	通常与 temperature 配合使用
top_k	限制候选词数量	过小可能使表达单一
repetition_penalty	降低重复片段概率	过高可能破坏语句自然度
max_new_tokens	限制新增令牌数量	按学段在配置文件中分别设置
seed	设置随机种子	用于同一环境下复现实验

5 文本质量评估

5.1 评估直接输入文本

```
llmctg evaluate --text "春天来了。小草从泥土里探出头。" --level primary
```

5.2 评估文本文件

```
llmctg evaluate --input article.txt --level junior --output output/evaluation.json
```

5.3 理解评估结果

指标	含义
overall_score	结构、长度、等级匹配和流畅性的加权总分
predicted_level	根据句长和字词难度估算的阅读等级
repetition_rate	连续四字片段的重复程度
english_ratio	英文字符占全文字符的比例
uncommon_character_ratio	不在内置常用字集合中的汉字比例
warnings	触发阈值的质量问题
suggestions	针对问题生成的修改建议

自动评估是编辑辅助工具，不能判断所有事实是否真实，也不能代替教师、编辑或学科专家的最终审核。

6 批量任务处理

6.1 准备任务文件

任务文件采用 JSONL，即每一行是一个独立 JSON 对象。主题 topic 必填，其他字段可省略。

```
{"topic":"森林里的水循环","level":"primary","keywords":["树叶","雨水"]}
{"topic":"数据如何影响公共决策","level":"senior","instruction":"说明证据边界"}
```

6.2 执行任务

```
llmctg batch --input examples/tasks.jsonl --output output/batch
```

默认启用断点续作。系统为每条源任务计算稳定编号，已完成任务会跳过。若需要重新计算，可使用 `--no-resume` 并选择新的输出目录。单条任务失败后，原因写入 `errors.jsonl`，其余任务继续执行。

6.3 批处理结果检查

1. 打开 `summary.json`，确认 `generated`、`passed`、`failed` 和 `average_score` 等汇总值符合任务规模。
2. 打开 `articles.jsonl`，逐行检查 `topic`、`level`、`content`、`backend` 和 `evaluation` 字段。
3. 若存在 `errors.jsonl`，根据 `line_number` 定位输入行，并根据 `error_type` 和 `message` 修正任务。
4. 使用浏览器打开 `report.html` 进行人工抽查；使用表格软件打开 `summary.csv` 进行筛选或汇总。

7 训练数据准备

原始文本每行由正文、制表符和等级标签构成。等级可使用 0、1、2 或小学、初中、高中。转换器会生成带 `instruction`、`input`、`output` 和 `level` 字段的 JSONL。

```
llmctg prepare-data --input train.txt --output data/train.jsonl
```

7.1 原始标注格式

每个非空行只能表示一个样本。正文与等级之间必须使用真正的 **TAB** 字符，而不是多个空格。等级 0 表示小学，1 表示初中，2 表示高中。转换过程中遇到缺少 **TAB**、正文为空或等级无法识别时，程序会报告具体行号。

7.2 转换结果核验

```
python -m json.tool --json-lines data/train.jsonl
```

每行应包含 `instruction`、`input`、`output` 和 `level`。`output` 保存训练正文，`level` 使用 `primary`、`junior` 或 `senior`。正式训练前建议检查样本是否含乱码、敏感信息、重复文本以及超出目标学段的内容。

8 LoRA 模型微调

微调前应确认基础模型和训练语料的授权，准备足够磁盘与显存，并先使用少量样本验证流程。软件使用显式参数替代原始脚本中的个人绝对路径。

```
CUDA_VISIBLE_DEVICES=0 llmctg train \  
  --model-path /home/wangzy/workplace/wyc/Qwen3-1.7B \  
  --dataset test_data/llmctg_smoke_train.jsonl \  
  --output test_output/qwen3-1.7b-lora-smoke \  
  --epochs 1 --batch-size 1 --max-length 256 --device cuda:0
```

- 训练输出目录保存 LoRA 适配器和分词器配置。

- CUDA 环境使用半精度并启用梯度检查点与输入梯度，以降低训练显存压力；CPU 环境使用单精度。
- 若模型层名称与 `q_proj`、`k_proj`、`v_proj`、`o_proj` 不匹配，应由维护人员调整目标模块。
- 训练损失只反映拟合过程，不能代替独立测试集和人工内容评价。

8.1 训练过程观察

训练启动后依次显示模型分片加载、数据映射和进度条。可在另一个终端运行 `nvidia-smi` 观察显存与利用率。12 条样本、单卡、批量大小 1、梯度累积 4 时通常约有 3 个优化步；具体步数可能随 Transformers 版本和分布式环境变化。

8.2 训练成功判据

1. 命令正常返回，结果 JSON 中包含 `output_dir`、`train_samples`、`train_loss`、`global_step` 和 `device`。
2. 输出目录存在 `adapter_config.json` 和 `adapter_model.safetensors`。
3. 分词器相关文件已保存，可以与基础模型共同用于推理。
4. 使用 `--lora-path` 完成至少一次真实生成，并确认 `content` 字段非空。

8.3 单卡与多卡说明

仅传入 `--device cuda:0` 不一定限制 Trainer 可见的显卡数量。小规模数据在多卡数据并行下可能因复制和同步开销而变慢。冒烟测试建议在命令前使用 `CUDA_VISIBLE_DEVICES=0`；正式训练再根据数据量和分布式训练方案决定是否使用多卡。

9 配置说明

执行 `init-config` 可生成默认 JSON 配置。命令行中的后端、模型路径和设备会覆盖配置文件对应值。

```
llmctg init-config --output llmctg.json
llmctg --config llmctg.json generate --topic "气候"
```

配置项	默认值	说明
<code>generation.backend</code>	<code>template</code>	生成后端

配置项	默认值	说明
generation.model_path	空	本地基础模型目录
generation.lora_path	空	可选 LoRA 适配器目录
generation.device	auto	推理设备
generation.temperature	0.9	采样随机性
generation.top_p	0.9	核采样阈值
generation.repetition_penalty	1.1	模型重复惩罚
evaluation.pass_score	60	质量通过分数
evaluation.repetition_warning	0.18	重复率警告阈值
log_level	INFO	日志级别

10 输出文件说明

JSON 和 JSONL 保存全部结构化信息，适合程序继续处理；CSV 以 UTF-8 BOM 编码，适合 Excel 等软件；HTML 报告完全本地化，可直接双击打开。生成时间采用带时区的 ISO 8601 格式。

11 一键功能测试

scripts/one_click_test.py 用于完成全功能验收。脚本采用预设 Linux 路径，按时间创建独立输出目录，逐项显示运行功能与预期结果，并在结束时写入 test_summary.json。某一项失败后，脚本会记录失败并继续执行后续独立项目。

```
cd /home/wangzy/workplace/wyc/llmctg
python scripts/one_click_test.py
```

测试序号	功能	主要成功判据
------	----	--------

测试序号	功能	主要成功判据
1	版本与命令入口	显示版本且退出码为 0
2	默认配置生成	JSON 包含 generation 和 evaluation
3	训练数据转换	生成至少 12 条有效 JSONL
4	模板文本生成	正文生成且 backend 为 template
5	文本质量评估	结果包含 metrics 和 statistics
6	批量任务处理	生成文章、摘要、CSV 和 HTML
7	完整离线演示	生成三个学段示例及报告
8	本地基础模型生成	Qwen3 输出非空正文
9	LoRA 冒烟微调	适配器配置和权重成功保存
10	LoRA 适配器生成	挂载适配器后输出非空正文

只检查不依赖大模型的基础功能时使用 `python scripts/one_click_test.py --skip-llm`。需要改变部署位置时，可传入 `--project-root`、`--model-path` 和 `--device`。所有结果位于 `test_output/one_click_时间戳`，脚本整体通过时退出码为 0。

12 Notebook 分批测试

`notebooks/llmctg_function_test.ipynb` 适合交互式验收。Notebook 把测试拆分为初始化、基础功能、生成评估、批处理、基础模型推理、LoRA 训练和 LoRA 推理等单元，每个单元都列出预期结果并通过 `assert` 自动判定关键输出。

```
cd /home/wangzy/workplace/wyc/llmctg
jupyter notebook notebooks/llmctg_function_test.ipynb
```

1. 选择安装 llmctg 及其模型扩展依赖的 Python 内核。
2. 先运行初始化单元，核对 PROJECT_ROOT、MODEL_PATH、DEVICE 和 OUTPUT_ROOT。

- 按从上到下的顺序运行；后续训练和 LoRA 单元依赖前面生成的 TRAIN_FILE 与 ADAPTER_PATH。
- 观察单元末尾是否显示 PASS；若 assert 失败，从当前单元的命令输出定位问题。
- 训练时可同时打开 nvidia-smi，确认只有指定 GPU 被当前进程使用。

13 常见问题与故障处理

现象	原因与处理方法
invalid choice: 城市河流	generate 子命令缺失或参数顺序错误；应先写 llamctg generate，再写 --topic
提示主题不能为空	检查 --topic 或批量任务 topic 字段，删除只有空格的值
提示不支持的阅读等级	使用 primary、junior、senior、0、1、2 或相应中文学段
提示缺少模型依赖	安装 [llm] 扩展，并确认当前命令使用同一 Python 环境
模型目录不存在	检查路径拼写与访问权限，不要填写网络模型名称代替本地目录
NVIDIA driver is too old	安装与驱动兼容的 PyTorch CUDA 构建，或由管理员升级驱动；先核对 torch.version.cuda
model_kwargs 包含 generator	更新到已移除 generator 参数的当前版本，再执行 pip install -e .
TrainingArguments 参数不兼容	更新当前代码；系统会按已安装 Transformers 的签名筛选可用参数
Unable to create tensor	更新当前代码；训练标签应由支持动态 padding 的数据整理器生成
训练只有 1 步且很慢	可能自动使用多卡；用 CUDA_VISIBLE_DEVICES=0 限制单卡后重试
找不到 adapter_config.json	--lora-path 必须指向完整适配器目录，不能指向基础模型或单个权重文件
CUDA 显存不足	降低 max-length、使用更小模型、减少批量大小或改用 CPU

现象	原因与处理方法
批量任务部分失败	查看 <code>errors.jsonl</code> 的行号、异常类型和消息，修正后使用新目录重跑
中文在终端显示异常	将终端切换到 UTF-8，文件本身仍按 UTF-8 保存
质量分低	查看 <code>warnings</code> 与 <code>suggestions</code> ，重点检查长度、长句、重复和等级匹配

14 数据安全与使用边界

- 默认后端完全离线，不向网络发送主题或文章。
- 本地模型后端只读取用户指定目录，但模型、语料和数据库授权由使用者负责确认。
- 请勿把含个人敏感信息、未授权作品或保密材料直接用于训练。
- 生成内容投入教学、出版或公开传播前，必须进行事实、版权、年龄适宜性和价值导向审核。
- 备份输出目录时应遵守所在单位的数据分类与保存制度。

15 日常维护与卸载

15.1 日常维护

- 升级代码后重新执行 `python -m pip install -e .`，并运行不依赖模型的快速测试。
- 不要把模型权重、训练检查点、环境变量文件和 `test_output` 提交到版本库；项目已提供 `.gitignore`。
- 定期归档正式训练的适配器、配置、数据版本和测试摘要，以便复现。
- 清理输出目录前先确认其中没有唯一的适配器权重或验收记录。

15.2 软件卸载

使用 `pip` 安装时执行下列命令卸载程序。输出目录、模型和训练数据不会自动删除，应由用户确认后单独归档或清理。

```
python -m pip uninstall llmctg
```

附录 A 命令速查

```
llmctg --version
llmctg init-config --output llmctg.json
llmctg demo --output output/demo
llmctg generate --backend template --topic "城市河流" --level junior
llmctg evaluate --input article.txt --level junior --output evaluation.json
llmctg batch --input examples/tasks.jsonl --output output/batch --no-resume
llmctg prepare-data --input train.txt --output train.jsonl
llmctg train --model-path MODEL --dataset train.jsonl --output ADAPTER --device cuda:0
llmctg generate --backend transformers --model-path MODEL --lora-path ADAPTER --device cuda:0 --topic "城市河流"
```

附录 B 验收检查表

检查项目	合格标准	结果
安装入口	llmctg --version 正常返回	☐
模板生成	输出正文与完整评估字段	☐
文本评估	JSON 包含统计、分数、警告和建议	☐
批量任务	正常记录成功项并隔离异常项	☐
报告导出	JSONL、JSON、CSV、HTML 可正常打开	☐
本地模型	指定 Qwen3 模型能够生成非空正文	☐
LoRA 训练	适配器配置与权重成功保存	☐
LoRA 推理	--lora-path 加载成功并生成正文	☐
一键测试	test_summary.json 中所有项目为 PASS	☐
Notebook	各批次单元按顺序执行通过	☐